

CSE3031 EXAM CHEAT SHEET: 10 HIGH-YIELD QUESTIONS AND ANSWERS

Source basis: HW1–HW4 + Midterm Review

Topic weighting used: 8/10 questions from Memory Hierarchy, Virtual Memory, Advanced Memory, Pipelining, ILP; 2/10 from Fundamentals/ISA/Basic Memory.

1) [Memory Hierarchy] How do you compute AMAT, and what does each term mean?

Answer:

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

- Hit time: time to access cache when hit occurs.
 - Miss rate: fraction of accesses that miss.
 - Miss penalty: extra time to fetch from lower memory level.
- Use AMAT to compare cache designs; lower AMAT is better.

2) [Memory Hierarchy] For a set-associative cache, how do you split address bits into Tag/Index/Offset?

Answer:

1. Compute #blocks = cache size / block size.
 2. Compute #sets = #blocks / associativity.
 3. Offset bits = $\log_2(\text{block size})$.
 4. Index bits = $\log_2(\text{\#sets})$.
 5. Tag bits = address bits – index bits – offset bits.
- This method is central to HW3 cache-design problems.

3) [Virtual Memory] What is the translation flow with TLB + page table, and when does a page fault occur?

Answer:

1. Use virtual page number (VPN) to check TLB.
 2. TLB hit: get physical frame number (PFN), combine with offset.
 3. TLB miss: consult page table.
 4. If page-table valid bit is set, translation succeeds (page hit).
 5. If valid bit is clear, page fault occurs; OS brings page from disk.
- TLB miss does not always mean page fault.

4) [Virtual Memory] How do FIFO and LRU differ in page replacement?

Answer:

- FIFO: evict page that entered memory earliest (oldest resident).
- LRU: evict page least recently used.

Given a reference string and limited frames, simulate frame contents step-by-step to count faults.

LRU uses recency; FIFO ignores recency.

5) [Advanced Memory] Why are L1 caches often split (I-cache + D-cache) while L2 is unified?

Answer:

- Split L1 reduces structural hazards and allows simultaneous instruction fetch + data access each cycle.
- Unified L2 improves capacity sharing and reduces total miss rate by dynamically sharing space

between instructions and data.

This reflects the latency-vs-capacity tradeoff across hierarchy levels.

6) [Pipelining] What is ideal pipeline speedup and how do stalls reduce it?

Answer:

- Ideal speedup for an N-stage balanced pipeline is approximately N (for long instruction streams).

- Real CPI becomes:

$$\text{CPI}_{\text{actual}} = \text{CPI}_{\text{ideal}} + \text{stall cycles per instruction}$$

Stalls from hazards increase CPI and reduce achieved speedup.

7) [Pipelining] How do you classify hazards and decide stall vs forwarding?

Answer:

- RAW (read-after-write): true dependency; common source of data hazards.

- WAR/WAW: typically not hazards in simple in-order 5-stage pipelines.

- Forwarding resolves many RAW hazards by bypassing results.

- Load-use RAW often still needs a stall because load data is ready too late.

- Control hazards (branches) may require flush/stall depending on predictor and resolution stage.

8) [ILP / Branch Prediction] How does a 2-bit saturating counter predictor work?

Answer:

States: 00 (SNT), 01 (WNT), 10 (WT), 11 (ST).

- Predict Taken in states 10/11.

- Predict Not Taken in states 00/01.

- On actual Taken: increment state (saturate at 11).

- On actual Not Taken: decrement state (saturate at 00).

It needs two consecutive opposite outcomes to flip a strong prediction, reducing noise vs 1-bit predictors.

9) [Fundamentals] What is the CPU performance equation and how is it used?

Answer:

$$\begin{aligned} \text{CPU Time} &= \text{Instruction Count} \times \text{CPI} \times \text{Cycle Time} \\ &= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}} \end{aligned}$$

Use it to compare implementations by changing IC, CPI, and clock rate. Performance improves when CPU time decreases.

10) [ISA] For fixed-length encoding, how do you test whether an instruction set can be encoded?

Answer:

Compute opcode-space usage by instruction format.

- With instruction length L and register field size R:

- 2-address instruction consumes 2^{L-2R} encoding space per opcode pattern.

- 1-address consumes 2^{L-R} space per opcode pattern.

- 0-address consumes 2^L space per opcode pattern.

A practical method: reserve bit patterns for larger-address formats first, then verify remaining

patterns can encode required 1-address and 0-address instructions.

Ultra-compact formula box

- CPU Time = IC × CPI × Cycle Time
- Amdahl: $\text{Speedup} = 1 / ((1-f) + f/s)$
- AMAT = Hit Time + Miss Rate × Miss Penalty
- Pipeline CPI = 1 + stall/instruction (ideal baseline)
- Tag bits = Addr bits – Index bits – Offset bits